Eine Web-Plattform auf einem Oracle-Cloud-Server betreiben

Schritt-für-Schritt-Anleitung: kostenloser Server („Always Free“), HTTPS, Dienst, Backups und automatische Updates

Tipp: Alle Namen, Adressen und Pfade in dieser Anleitung sind **Platzhalter**. Ersetze sie durch deine eigenen Werte: `meine-app` = Name deiner Anwendung, `203.0.113.10` = öffentliche IP deines Servers, `3000` = interner Port der Anwendung, `https://example.com/…` = dein Code-Repository.

Inhalt

1 Überblick: So hängt alles zusammen
2 Voraussetzungen
3 Oracle-Konto anlegen
4 Server erstellen
5 Ports freigeben
6 Per SSH verbinden
7 Server vorbereiten
8 Anwendung holen und bauen
9 Anwendung als Dienst einrichten
10 HTTPS mit eigener Adresse
11 Geheimnisse sicher ablegen
12 Backups
13 Updates
14 Wartung und Fehlersuche
15 Gut zu wissen
16 Checkliste

Beispielhaft geht die Anleitung von einer Node.js-Anwendung mit einer SQLite-Datenbank aus. Für andere Laufzeiten (Python, Go, Java …) ändern sich nur die Schritte „Laufzeit installieren“ und „Anwendung bauen“ – der Rest bleibt gleich.

1. Überblick: So hängt alles zusammen

Eine Anfrage aus dem Internet passiert mehrere Stationen, bis sie bei der Anwendung ankommt. Jede davon muss richtig eingestellt sein – die meisten Probleme entstehen, wenn eine Station fehlt.

Station	Aufgabe	Wo eingestellt
Adresse (Domain)	Name statt IP, nötig für das HTTPS-Zertifikat	z. B. sslip.io, DuckDNS oder eigene Domain
Oracle-Firewall (Security List)	Lässt Ports 80/443 überhaupt bis zum Server durch	Oracle-Konsole
Server-Firewall (iptables)	Zweite Firewall direkt auf dem Server	Kommandozeile
Webserver mit HTTPS (Caddy)	Holt automatisch ein Zertifikat, leitet an die App weiter	/etc/caddy/Caddyfile
Anwendung (Dienst)	Läuft dauerhaft, startet bei Absturz/Neustart neu	systemd-Unit
Daten	Datenbank und hochgeladene Dateien	eigener Datenordner + tägliches Backup

Die Anwendung selbst lauscht nur auf `127.0.0.1:3000` – also nur innerhalb des Servers. Von außen erreichbar ist ausschließlich der Webserver auf den Ports 80 und 443. So muss sich die Anwendung nicht selbst um Zertifikate kümmern.

2. Voraussetzungen

- Eine E-Mail-Adresse und eine Kreditkarte (nur zur Identitätsprüfung – bei reinen „Always Free“-Ressourcen wird nichts abgebucht).
- Ein PC mit Terminal: unter Windows **PowerShell**, unter macOS/Linux das **Terminal**. SSH ist dort bereits enthalten.
- Der Code deiner Anwendung in einem Git-Repository (z. B. GitHub, GitLab) – öffentlich oder mit Zugangsschlüssel.
- Etwa 30–60 Minuten Zeit.

3. Oracle-Konto anlegen

1. Auf der Oracle-Cloud-Seite „Free Tier“ bzw. „Kostenlos starten“ wählen.
2. **Home-Region** aussuchen – möglichst nah an deinen Nutzern (z. B. Frankfurt für Deutschland). **Die Region lässt sich später nicht mehr ändern**, und Always-Free-Server gibt es nur in der Home-Region.
3. Kreditkarte zur Prüfung angeben und das Konto bestätigen. Die Freischaltung kann einige Minuten dauern.

4. Server erstellen

1. In der Konsole: ☰ → **Compute → Instances → Create instance**.
2. **Name** frei wählen, z. B. `meine-app`.

3. **Image:** „Change image“ → **Canonical Ubuntu** (aktuelle LTS-Version, z. B. 24.04).
4. **Shape:** „Change shape“ → eine mit **„Always Free-eligible“** gekennzeichnete Größe, z. B. `VM.Standard.E2.1.Micro` (AMD, 1 GB RAM) oder – falls verfügbar – `VM.Standard.A1.Flex` (Ampere/ARM, bis 4 Kerne / 24 GB im Gratiskontingent).
5. **Networking:** „Create new virtual cloud network“ und **„Assign a public IPv4 address“** aktiviert lassen.
6. **SSH keys:** „Generate a key pair“ → **„Save private key“**. Die Datei sicher aufbewahren, z. B. unter `~/.ssh/meine-app.key`. Ohne sie kommst du nicht auf den Server.
7. **Create** klicken. Nach 1–2 Minuten steht die Instanz auf „Running“. Die **Public IP address** notieren.

> **Tipp:** Ampere-Server (A1) sind oft ausgebucht („Out of capacity“). Dann einfach später erneut versuchen oder die kleine AMD-Variante nehmen – für eine Plattform mit wenigen Nutzern reicht sie gut.

5. Ports freigeben (Oracle-Firewall)

Standardmäßig lässt Oracle nur SSH (Port 22) durch. Für eine Webseite braucht es zusätzlich 80 (HTTP, für das Zertifikat) und 443 (HTTPS).

1. Auf der Instanzseite unter **Primary VNIC** auf das **Subnet** klicken.
2. **Security Lists** (bzw. „Security“) → **Default Security List** öffnen.
3. **Add Ingress Rules** und zwei Regeln anlegen:

Source CIDR	IP Protocol	Destination Port Range
`0.0.0.0/0`	TCP	80
`0.0.0.0/0`	TCP	443

> **Achtung:** Das ist nur die **erste** von zwei Firewalls. Die zweite sitzt auf dem Server selbst (Schritt 7.4) – beide müssen offen sein.

6. Per SSH verbinden

Windows (PowerShell):

```
ssh -i $HOME\.ssh\meine-app.key ubuntu@203.0.113.10
```

Meldet SSH „UNPROTECTED PRIVATE KEY FILE“, die Dateirechte einmal einschränken:

```
icacls $HOME\.ssh\meine-app.key /inheritance:r /grant:r "$($env:USERNAME):R"
```

macOS / Linux:

```
chmod 600 ~/.ssh/meine-app.key
ssh -i ~/.ssh/meine-app.key ubuntu@203.0.113.10
```

Die Frage „Are you sure you want to continue connecting?“ beim ersten Mal mit yes beantworten. Der Benutzer heißt bei Ubuntu-Images von Oracle `ubuntu`.

7. Server vorbereiten

7.1 System aktualisieren

```
sudo apt-get update && sudo apt-get -y upgrade
sudo timedatectl set-timezone Europe/Berlin     # eigene Zeitzone wählen
```

7.2 Auslagerungsspeicher (Swap) anlegen

Die kleinen Gratis-Server haben nur 1 GB Arbeitsspeicher. Beim Bauen der Anwendung reicht das oft nicht – 2 GB Swap verhindern Abstürze.

```
sudo fallocate -l 2G /swapfile && sudo chmod 600 /swapfile
sudo mkswap /swapfile && sudo swapon /swapfile
echo '/swapfile none swap sw 0 0' | sudo tee -a /etc/fstab
```

7.3 Pakete und Laufzeit installieren

```
sudo apt-get install -y curl git sqlite3 ca-certificates gnupg

# Node.js (Beispiel: Version 22) aus den offiziellen Paketquellen
curl -fsSL https://deb.nodesource.com/setup_22.x | sudo bash -
sudo apt-get install -y nodejs

# Caddy – Webserver mit automatischem HTTPS
sudo apt-get install -y debian-keyring debian-archive-keyring apt-transport-https
curl -1sLf https://dl.cloudsmith.io/public/caddy/stable/gpg.key \
  | sudo gpg --dearmor -o /usr/share/keyrings/caddy-stable-archive-keyring.gpg
curl -1sLf https://dl.cloudsmith.io/public/caddy/stable/debian.deb.txt \
  | sudo tee /etc/apt/sources.list.d/caddy-stable.list
sudo apt-get update && sudo apt-get install -y caddy
```

7.4 Server-Firewall öffnen

Die Ubuntu-Images von Oracle blockieren per `iptables` alles außer SSH. Die neuen Regeln müssen **vor** der vorhandenen REJECT-Regel stehen, sonst greifen sie nie:

```
POS=$(sudo iptables -L INPUT --line-numbers -n | awk '$2=="REJECT"{print $1; exit}')
sudo iptables -I INPUT $POS -p tcp --dport 80  -m state --state NEW -j ACCEPT
sudo iptables -I INPUT $POS -p tcp --dport 443 -m state --state NEW -j ACCEPT
sudo apt-get install -y iptables-persistent
sudo netfilter-persistent save                # Regeln über Neustarts behalten
```

> **Achtung:** Nicht `ufw` und die Oracle-iptables-Regeln mischen – das führt leicht dazu, dass man sich selbst aussperrt. SSH (Port 22) nie schließen.

8. Anwendung holen und bauen

Code nach `/opt/meine-app` laden, Abhängigkeiten installieren, bauen und – wenn vorhanden – die Tests laufen lassen:

```
sudo git clone --depth 1 --branch main https://example.com/benutzer/meine-app.git /opt/meine-app
cd /opt/meine-app
sudo npm ci                 # exakt die Versionen aus der Lock-Datei
sudo npm run build          # z. B. Oberfläche bauen
sudo npm test               # optional, aber empfohlen
```

Bei einem privaten Repository einen schreibgeschützten Zugangsschlüssel („Deploy Key“ bzw. „Read-only Token“) verwenden – nie das eigene Passwort auf dem Server speichern.

9. Anwendung als Dienst einrichten

Ein eigener Systembenutzer ohne Login sorgt dafür, dass die Anwendung nur auf ihren Datenordner schreiben darf. **systemd** startet sie beim Hochfahren und nach einem Absturz automatisch neu.

```
sudo useradd --system --home /var/lib/meine-app --shell /usr/sbin/nologin meine-app
sudo install -d -o meine-app -g meine-app -m 750 /var/lib/meine-app
```

Datei `/etc/systemd/system/meine-app.service` anlegen (z. B. mit `sudo nano …`):

```
[Unit]
Description=Meine App
After=network.target

[Service]
User=meine-app
WorkingDirectory=/opt/meine-app
Environment=NODE_ENV=production
Environment=PORT=3000
Environment=HOST=127.0.0.1
Environment=DATABASE_FILE=/var/lib/meine-app/app.db
Environment=TZ=Europe/Berlin
ExecStart=/usr/bin/node server/index.js
Restart=always
RestartSec=3
# Absicherung: nur der Datenordner ist beschreibbar
NoNewPrivileges=true
ProtectSystem=strict
ProtectHome=true
ReadWritePaths=/var/lib/meine-app
PrivateTmp=true

[Install]
WantedBy=multi-user.target
```

```
sudo systemctl daemon-reload
sudo systemctl enable --now meine-app
systemctl status meine-app                        # sollte "active (running)" zeigen
curl -s http://127.0.0.1:3000/                     # antwortet die App lokal?
```

> **Tipp:** Eine kleine „Health“-Adresse in der App (z. B. `/api/health`, die nur „ok“ zurückgibt) macht das Prüfen nach Updates und Neustarts viel einfacher.

10. HTTPS mit eigener Adresse

Für ein Zertifikat braucht der Server einen Namen. Drei Möglichkeiten:

Variante	Beispiel	Aufwand
sslip.io – Name direkt aus der IP	`203-0-113-10.sslip.io`	keiner, sofort nutzbar, kostenlos
DuckDNS o. ä. – kostenlose Subdomain	`meine-app.duckdns.org`	kurz registrieren, auf die IP zeigen lassen
Eigene Domain	`app.example.com`	Domain kaufen, A-Eintrag auf die IP setzen

Datei `/etc/caddy/Caddyfile` ersetzen:

```
203-0-113-10.sslip.io {
    encode gzip
    reverse_proxy 127.0.0.1:3000
}
```

```
sudo systemctl reload caddy
journalctl -u caddy -f            # Zertifikat wird beim ersten Aufruf geholt
```

Jetzt im Browser `https://203-0-113-10.sslip.io` öffnen. Der erste Aufruf kann 1–2 Minuten dauern, bis das Zertifikat ausgestellt ist. Caddy erneuert es danach automatisch.

> **Achtung:** Klappt das Zertifikat nicht, liegt es fast immer an den Ports: Port 80 muss in **beiden** Firewalls offen sein (Schritte 5 und 7.4).

11. Geheimnisse sicher ablegen

API-Schlüssel, Zugangsdaten für Fremddienste usw. gehören **nie** ins Git-Repository. Besser in einen eigenen Ordner, den nur root und der Dienstbenutzer lesen dürfen:

```
sudo install -d -o root -g meine-app -m 750 /etc/meine-app
sudo nano /etc/meine-app/dienst.json             # Inhalt einfügen, speichern
sudo chown root:meine-app /etc/meine-app/dienst.json
sudo chmod 640 /etc/meine-app/dienst.json
```

In der systemd-Unit dann z. B. `Environment=SERVICE_KEY_FILE=/etc/meine-app/dienst.json` setzen und die Datei in der Anwendung beim Start lesen. Schlüssel nicht über Chats, Tickets oder E-Mails weitergeben – direkt auf dem Server einfügen.

12. Backups

Ein tägliches Backup der Datenbank, 14 Tage aufbewahrt. SQLite bietet dafür einen eigenen, im laufenden Betrieb sicheren Befehl:

```
sudo install -d -m 750 /var/backups/meine-app
sudo tee /etc/cron.daily/meine-app-backup >/dev/null <<'EOF'
#!/bin/sh
sqlite3 /var/lib/meine-app/app.db ".backup '/var/backups/meine-app/app-$(date +%F).db'"
find /var/backups/meine-app -name 'app-*.db' -mtime +14 -delete
EOF
sudo chmod 755 /etc/cron.daily/meine-app-backup
```

Hochgeladene Dateien (Bilder usw.) zusätzlich sichern, z. B. mit `tar czf` des Upload-Ordners. Und ab und zu ein Backup auf den eigenen PC holen – liegt es nur auf dem Server, ist es mit ihm weg:

```
scp -i ~/.ssh/meine-app.key "ubuntu@203.0.113.10:/var/backups/meine-app/*.db" .
```

> **Tipp:** Ein Backup ist erst gut, wenn man das Zurückspielen einmal ausprobiert hat: Dienst stoppen, Datei an die Stelle der Datenbank kopieren, Rechte für den Dienstbenutzer setzen, Dienst starten.

13. Updates

13.1 Von Hand

```
cd /opt/meine-app
sudo git pull
sudo npm ci && sudo npm run build && sudo npm test
sudo systemctl restart meine-app
```

13.2 Automatisch jede Nacht

Ein kleines Skript prüft, ob es Neues gibt, baut und testet – und startet nur neu, wenn alles geklappt hat. Schlägt etwas fehl, läuft die bisherige Version einfach weiter.

```
#!/usr/bin/env bash
# /opt/meine-app/update.sh
set -euo pipefail
cd /opt/meine-app
OLD=$(git rev-parse HEAD)
git fetch --quiet origin main
NEW=$(git rev-parse FETCH_HEAD)
[ "$OLD" = "$NEW" ] && { echo "Keine Änderungen"; exit 0; }
git checkout --quiet -B main "$NEW"
if npm ci && npm run build && npm test; then
  systemctl restart meine-app
else
  git checkout --quiet -B main "$OLD"       # zurück auf den alten Stand
  npm ci && npm run build
  exit 1
fi
```

Dazu eine systemd-Unit und einen Timer:

```
# /etc/systemd/system/meine-app-update.service
[Unit]
Description=Meine App aktualisieren
After=network-online.target
[Service]
Type=oneshot
ExecStart=/bin/bash /opt/meine-app/update.sh

# /etc/systemd/system/meine-app-update.timer
[Unit]
Description=Meine App jede Nacht aktualisieren
[Timer]
OnCalendar=*-*-* 03:30:00
RandomizedDelaySec=20min
Persistent=true
[Install]
WantedBy=timers.target
```

```
sudo systemctl daemon-reload
sudo systemctl enable --now meine-app-update.timer
systemctl list-timers | grep meine-app           # nächster Lauf
```

13.3 Optional: Update auf Knopfdruck aus der Anwendung

Der Dienst selbst sollte keine Root-Rechte haben. Ein sicherer Weg: Die Anwendung legt nur eine Datei an (z. B. `/var/lib/meine-app/update-request`). Eine systemd-**Path**-Unit bemerkt die Datei und startet den Update-Dienst als root. Das Update-Skript löscht die Datei und schreibt seinen Fortschritt in eine Statusdatei, die die Anwendung anzeigen kann.

```
# /etc/systemd/system/meine-app-update.path
[Path]
PathExists=/var/lib/meine-app/update-request
Unit=meine-app-update.service
[Install]
WantedBy=multi-user.target
```

14. Wartung und Fehlersuche

Was	Befehl
Läuft die App?	`systemctl status meine-app`
Protokoll live ansehen	`journalctl -u meine-app -f`
Neu starten	`sudo systemctl restart meine-app`
Webserver/Zertifikat prüfen	`journalctl -u caddy -n 50`
Antwortet die App intern?	`curl -s http://127.0.0.1:3000/`
Ports offen? (vom eigenen PC)	`curl -I http://203.0.113.10`
Speicherplatz / RAM	`df -h` · `free -h`
Letztes Update-Protokoll	`journalctl -u meine-app-update -n 50`

Problem	Häufige Ursache
Seite lädt gar nicht	Port 80/443 in einer der beiden Firewalls zu; Caddy läuft nicht
„502 Bad Gateway“	Webserver läuft, die App aber nicht (Protokoll der App ansehen) oder falscher Port im Caddyfile
Kein Zertifikat	Name zeigt nicht auf die IP, oder Port 80 ist von außen nicht erreichbar
Bauen bricht ab („Killed“)	Zu wenig Arbeitsspeicher – Swap anlegen (Schritt 7.2)
Dienst startet nicht	Rechte: Datenordner gehört nicht dem Dienstbenutzer; Pfad in `ExecStart` falsch

15. Gut zu wissen

- **Ungenutzte Gratis-Server:** Oracle kann Always-Free-Instanzen zurückfordern, die über längere Zeit kaum ausgelastet sind. Wer auf Nummer sicher gehen will, stellt das Konto auf „Pay As You Go“ um – Always-Free-Ressourcen bleiben kostenlos, diese Regel entfällt aber. Unabhängig davon: regelmäßig Backups auf den eigenen PC holen.
- **Kosten im Blick:** Nur Ressourcen mit dem Hinweis „Always Free-eligible“ nutzen; ein Budget mit E-Mail-Warnung in der Konsole schadet nicht.
- **Feste IP:** Die öffentliche IP bleibt normalerweise, solange die Instanz besteht. Wird die Instanz neu erstellt, ändert sie sich – dann Domain/DNS anpassen (bei sslip.io ändert sich der Name mit).
- **Sicherheit:** SSH nur mit Schlüssel (Standard bei Oracle), System regelmäßig aktualisieren (`sudo apt-get upgrade`), Anwendung nicht als root laufen lassen.
- **Ein Installationsskript**, das alle Schritte ab 7 enthält und gefahrlos mehrfach laufen kann, spart später viel Zeit – damit wird auch das Aktualisieren und ein Umzug auf einen neuen Server zum Einzeiler.

16. Checkliste

- ☐ Konto angelegt, Home-Region passend gewählt
- ☐ Instanz „Running“, öffentliche IP notiert, privater SSH-Schlüssel gesichert
- ☐ Security List: TCP 80 und 443 eingehend erlaubt
- ☐ Per SSH verbunden
- ☐ System aktualisiert, Zeitzone gesetzt, Swap angelegt
- ☐ Laufzeit und Caddy installiert
- ☐ Server-Firewall (iptables) für 80/443 geöffnet und gespeichert
- ☐ Code geholt, gebaut, Tests grün
- ☐ Dienstbenutzer + systemd-Unit eingerichtet, Dienst läuft
- ☐ Caddyfile mit Adresse eingerichtet, HTTPS funktioniert im Browser
- ☐ Geheimnisse unter /etc/meine-app mit Rechten 640
- ☐ Tägliches Backup eingerichtet und einmal zurückgespielt
- ☐ Automatische Updates (Timer) aktiv
- ☐ Backup zum ersten Mal auf den PC geholt